



PracticeInt

🕒 Created time	July 31, 2026 11:00 AM
☰ Category	CommonVariables
👤 Last edited by	 R.A R.A
🕒 Last updated time	August 1, 2026 6:13 AM

תרגול מעשי: עבודה עם Int

חלק 1: הצהרה והשמה (Declaration & Assignment)

1. צרי משתנה מסוג מספר שלם שייצג כמות של לקוחות במערכת Web שבנית.
2. הציבי בתוך המשתנה הזה ערך התחלתי כלשהו (מספר חיובי שלם).

חלק 2: מניפולציות ופעולות חשבון

1. צרי משתנה חדש שיחשב וישמור כמה לקוחות יהיו במערכת אם נכפיל את הכמות הנוכחית פי 4.
2. כעת, קחי את המשתנה המקורי (מהסעיף הראשון) ועדכני אותו כך שיוסיף לעצמו עוד 250 לקוחות חדשים. (יש לעדכן את המשתנה הקיים על סמך עצמו, ולא לדרוס אותו סתם עם מספר חדש).

חלק 3: המרה (Types Don't Cross)

1. הדפיסי לקונסול (Console) את כמות הלקוחות המעודכנת.
2. בצעי המרה מפורשת של המספר למחרוזת (String) כפי שלמדנו, כהכנה לשילוב של הנתון הזה בתצוגת ה-Front End.

חלק 4: מלכודת החילוק (The Division Trap)

1. חלקי את כמות הלקוחות המעודכנת שלך ב-3, והדפיסי את התוצאה.
2. הוסיפי הערה (Comment) בקוד שלך שמסבירה בקצרה: למה צריך להיזהר מאוד מחילוק של משתני Int? מה יקרה לתוצאה במקרה כזה?

חלק 5: מאחורי הקלעים (Bits & Signed Int32)

1. הוסיפי הערה בקוד המסבירה ב-2 משפטים: למה אנחנו משתמשים ב-Int32 (שהוא Signed) כברירת מחדל ב-C#? מה המשמעות של ה-"32" לגבי כמות ה-Bits, ומה הכוונה ב-Signed?

```
//Practical Exercise: Working with Int
//Part 1: Declaration & Assignment
//Create an integer variable that will represent the number
of customers in the Web system you built.
//Set some initial value (a positive integer) inside this v
ariable.
```

```
int totalCustomers = 10000;
```

```
//Part 2: Manipulations and Calculations
//Create a new variable that will calculate and store how m
any customers there will be in the system if we multiply th
e current number by 4.
```

```
int projectedCustomerCount = totalCustomers * 4;
```

```
//Now, take the original variable (from the first section)
and update it so that it adds another 250 new customers.
```

```
totalCustomers += 250;
```

```
//(The existing variable should be updated based on itself,
and not just overwritten with a new number).
```

```
//Part 3: Conversion (Types Don't Cross)
//Print the updated number of customers to the Console.
```

```
Console.WriteLine(totalCustomers);
```

```
//Perform an explicit conversion of the number to a String
as we learned, in preparation for integrating this data int
o the Front End display.
```

```
Console.WriteLine(totalCustomers.ToString());
```

```
//Part 4: The Division Trap
```

```
//Divide your updated number of customers by 3, and print the result.
```

```
totalCustomers /= 3;
```

```
Console.WriteLine(totalCustomers);
```

```
//Add a comment to your code that briefly explains: Why should you be very careful when dividing Int variables?
```

```
//What happens to the result in this case?
```

```
//Since int doesn't know how to handle decimal remainders, it simply truncates them, so we need to be very careful when dividing Integers.
```

```
//Part 5: Behind the Scenes (Bits & Signed Int32)
```

```
//Add a comment to your code that explains in 2 sentences:
```

```
Why do we use Int32 (which is Signed) by default in C#?
```

```
//Because it can hold about 2 billion in the positive direction and about 2 billion in the negative direction.
```

```
//And that's what we need in the vast majority of cases.
```

```
//What does the "32" mean in terms of Bits,
```

```
//32 bits, that's a huge amount of space to store ints.
```

```
//and what does Signed mean?
```

```
//The word signed means it has a plus or minus sign.
```

תרגול אתגר: מקרי קצה, מגבלות זיכרון ומלכודות לוגיות

חלק 1: התנגשות בגבולות הזיכרון (Int32 Limits)

1. צרי משתנה המייצג כמות בקשות לשרת (`serverRequests`) והציבי בו את המספר 2,000,000,000 (שני מיליארד - קרוב מאוד לגבול שלמדנו עליו).

2. קמפיין שיווקי גרם לתעבורה באתר להיות כפולה. צרי משתנה חדש שיחשב את התעבורה הכפולה (על ידי הכפלת המשתנה הראשון ב-2).
3. **בהערה (Comment) בקוד עני:** על בסיס התיאוריה שלמדנו על `Int32` וגודל המספרים שהוא יכול להכיל, מה יקרה לדעתך כשנגנסה להכניס את תוצאת ההכפלה למשתנה החדש? באיזה סוג משתנה (שהוזכר מפורשות בשיעור) היינו צריכים להשתמש פה במקום?

חלק 2: אשליית החילוק (The Division Trap in Action)

1. צרי משתנה בשם `totalPixels` (מייצג רוחב מסך בתצוגת Front End) והציבי בו את הערך `1000`.
2. אנחנו רוצים לחלק את רוחב המסך ל-3 עמודות שוות. צרי משתנה בשם `pixelsPerColumn` וחלקי בו את ה- `totalPixels` ב-3.
3. כעת, צרי משתנה שלישי בשם `restoredTotal`. קחי את `pixelsPerColumn` והכפילי אותו בחזרה ב-3 (כדי לבדוק אם נחזור לרוחב המקורי).
4. המירי את `restoredTotal` למחרוזת (String) בשביל תצוגה והדפיסי לקונסול.
5. **בהערה (Comment) בקוד עני:** האם הערך המודפס יחזור להיות בדיוק 1000? הסבירי במדויק למה, תוך שימוש בחוקים שלמדנו על התנהגות Integers.

חלק 3: לוגיקה בינארית (Binary Multiplication)

- הוסבר בשיעור ש-bit 1 מחזיק 2 מספרים, 2 bits מחזיקים 4 מספרים (00, 01, 10, 11), ו-3 bits מחזיקים 8 מספרים (0 עד 7).
1. **בהערה (Comment) בקוד עני:** לפי "חוק ההכפלה" (Multiplication) שטים קורי מסביר בחומר (כשמוסיפים עוד 0), כמה מספרים סך הכל יוכלו להחזיק **4 bits**?

```
//Part 1: Int32 Limits
//1. Create a variable that represents the number of requests to the server (`serverRequests`) and set it to 2,000,000,000 (two billion - very close to the limit we learned about).

UInt32 serverRequests = 2000000000;

//2. A marketing campaign caused the traffic on the site to double.
//Create a new variable that will calculate the double traffic (by multiplying the first variable by 2).
```

```
UInt32 trafficMultiple = serverRequests * 2;
```

//3. In a comment in the poor code: Based on the theory we learned about `Int32` and the size of numbers it can hold, what do you think will happen when we try to put the result of the multiplication into the new variable?

```
Console.WriteLine(trafficMultiple);  
//we get 4000000000
```

//What type of variable (that was explicitly mentioned in the lesson) should we have used here instead?
//UInt32

//Part 2: The Division Trap in Action

//1. Create a variable called `totalPixels` (representing the width of the screen in the Front End view) and set it to `1000`.

```
int totalPixels = 1000;
```

//2. We want to divide the width of the screen into 3 equal columns.

//Create a variable called `pixelsPerColumn` and divide the `totalPixels` by 3.

```
int pixelPerColumn = totalPixels / 3;  
Console.WriteLine(pixelPerColumn);
```

//3. Now, create a third variable called `restoredTotal`.
//Take `pixelsPerColumn` and multiply it back by 3 (to check if we get back to the original width).

```
int restoredTotal = pixelPerColumn * 3;
```

//4. Convert `restoredTotal` to a String for display and print to the console.

```
Console.WriteLine(restoredTotal.ToString());
```

//5. In a comment in the poor code: Will the printed value return to exactly 1000?

//Explain precisely why, using the rules we learned about the behavior of Integers.

//No, we get 999 .

//Part 3: Binary Logic (Binary Multiplication)

//It was explained in class that 1 bit holds 2 numbers, 2 bits hold 4 numbers (00, 01, 10, 11), and 3 bits hold 8 numbers (0 to 7).

//1. In a comment in poor code: According to the "Law of Multiplication" that Tim Corey explains in the material (when adding another 0), how many numbers in total can 4 bits hold?

// 16 numbers.

// Explanation: Every time we add a bit, we double the amount of numbers we can hold.

//If 3 bits hold 8 numbers, multiplying by 2 means 4 bits can hold 16 numbers.

תרגול מעשי: מניפולציות מתקדמות על Int

חלק 1: מ-String ל-Int (מטען ה-API / The API Payload)

באפליקציות ווב מודרניות, נתונים מגיעים לעיתים קרובות מה-Front-End כ-**string**. יש לנו מחרוזת המייצגת את כמות הלייקים על פוסט:

```
string likesFromFrontEnd = "8500";
```

1. בצע **Parse** ל-**string** הזה למשתנה **int** חדש בשם 'parsedLikes'.

חלק 2: אופרטור המודולו (%) - טיפול בשארית
ראית שחלוקת **Int** קוטעת את הנקודה העשרונית.

אבל מה אם אנחנו חייבים לדעת מה נשאר?
יש לנו 100 משימות רקע ו-3 שרתים לעבד אותן.

```
int totalTasks = 100;  
int serverCount = 3;
```

2. השתמשי באופרטור המודולו (%) כדי לחשב כמה משימות יישארו לאחר חלוקתן שווה בשווה בין 3 השרתים.
אחסני את התוצאה במשתנה בשם 'leftoverTasks' והדפיסי אותו.

חלק 3: זיכרון שריר - קידום והפחתה
בשנת 2026, אנחנו לא כותבים

```
variable = variable + 1  
או אפילו לספירה פשוטה  
variable += 1  
int activeConnections = 50;
```

3. משתמש חדש התחבר הרגע.

הגדילי את 'activeConnections' בדיוק ב-1 בעזרת התחביר הקצר והמקצועי ביותר שאפשר.

4. משתמש התנתק לפתע.

הפחיתי את המשתנה בדיוק ב-1 בעזרת התחביר הקצר ביותר.

חלק 4: Casting - המרות כפויות
מודל AI ניתח את המערכת שלנו והחזיר ציון ביטחון כ-'double'.

```
double aiConfidenceScore = 98.9;
```

5. אנחנו צריכים לשמור את זה ב-DB שלנו כ-Int פשוט.
בצעי Explicit Cast כדי לאלץ את ה-'double' הזה אל תוך משתנה 'int' חדש בשם 'finalScore'.

6. הוסיפי הערה המסבירה: מה קורה ל-9?
האם ה-Cast מעגלת את המספר למעלה ל-99, או קוטעת אותו ל-98?

```
//Practical Exercise: Advanced Int Manipulations
//Part 1: From String to Int (The API Payload)
//In modern web applications, data often comes from the front end as a string.
//We have a string representing the number of likes on a post:
```

```
string likesFromFrontEnd = "8500";
```

```
//Parse this string into a new int variable called 'parsedLikes'.
```

```
int parsedLikes = int.Parse(likesFromFrontEnd);
Console.WriteLine(parsedLikes);
```

```
//Part 2: The Modulo Operator (%) - Handling Remainder
//You saw that dividing Int truncates the decimal point.
//But what if we need to know what is left over?
//We have 100 background tasks and 3 servers to process them.
```

```
int totalTasks = 100;
```

```
int serverCount = 3;
```

```
//2. Use the modulo operator (%) to calculate how many tasks are left after dividing them equally among the 3 servers.
//Store the result in a variable called 'leftoverTasks' and print it.
```

```
int leftoverTasks = totalTasks % serverCount;
Console.WriteLine(leftoverTasks);
```

```
//Part 3: Muscle Memory - Promotion and Decrement
//In 2026, we don't write
```



```

variable = variable + 1
//or even for simple counting
variable += 1
int activeConnections = 50;

//3. A new user just connected.
//Increment 'activeConnections' by exactly 1 using the shortest syntax possible.

activeConnections++;
Console.WriteLine(activeConnections);

//4. A user suddenly disconnected.
//Decrement the variable by exactly 1 using the shortest syntax possible.

activeConnections--;
Console.WriteLine(activeConnections);

```

```

//Part 4: Casting - Forced Conversions
//The AI model analyzed our system and returned a confidence score as a 'double' .
double aiConfidenceScore = 98.9;

//5. We need to store this in our DB as a simple Int.
//Do an Explicit Cast to force this 'double' into a new 'int' variable called 'finalScore'.

int finalScore = (int)aiConfidenceScore;
Console.WriteLine(finalScore);

//6. Add an explanatory note: What happens to the .9?
//Does the Cast round the number up to 99, or truncate it to 98?
//The Cast operation does not round the number to 99.
//It simply truncates everything after the decimal point.
//So the result will be 98.

```